TEMA 2 · MAPA

Concurrencia: de «no pisarse» a «coordinarse»

Tema 1: los camareros solo tenían que no pisarse (synchronized). Tema 2: además tienen que esperarse, turnarse, avisarse y devolverse resultados.

wait / notifyAll

«Si no hay nada, me duermo y suelto; cuando cambie algo, me avisan.»

pág. 2 · C01

Pool de hilos

«Plantilla fija de camareros que se turnan el trabajo.»

pág. 3 · C02

Callable + Future

«Pido, me dan un ticket y con él recojo el resultado.»

págs. 4-5 · C02 · WriteCSV

AtomicInteger

«Leer, sumar y escribir de un golpe.»

pág. 6 · C03

ReentrantLock

«Un cerrojo con paciencia limitada (tryLock).»

pág. 6 · C03

Interbloqueo

«Todos cogen las cosas en el mismo orden.»

pág. 7 · C04

Semaphore

«Solo caben N a la vez.»

pág. 8 · C05

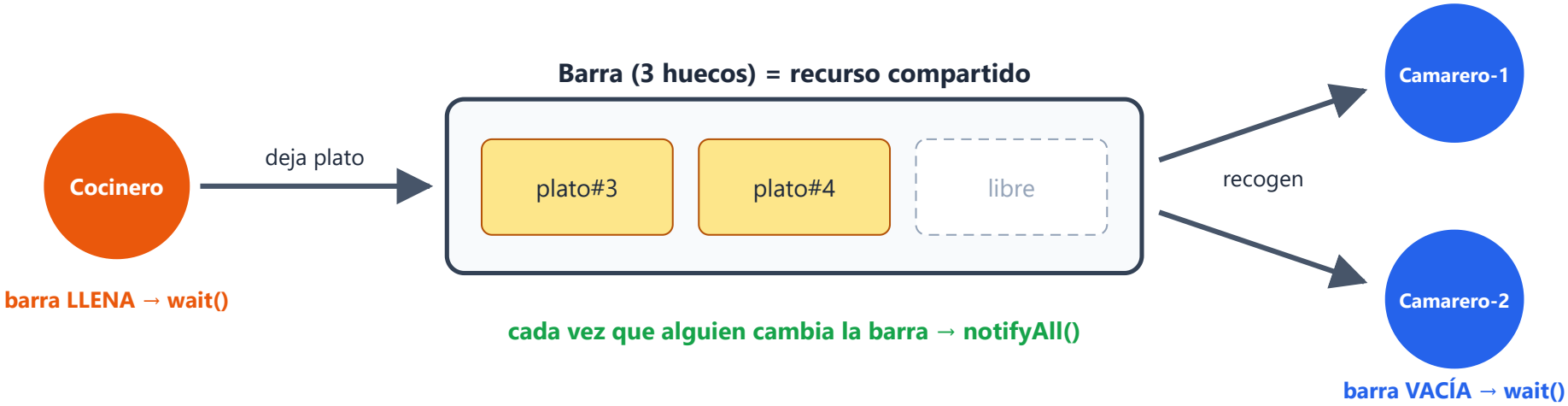
CountDownLatch

«No empiezo hasta que lleguen todos.»

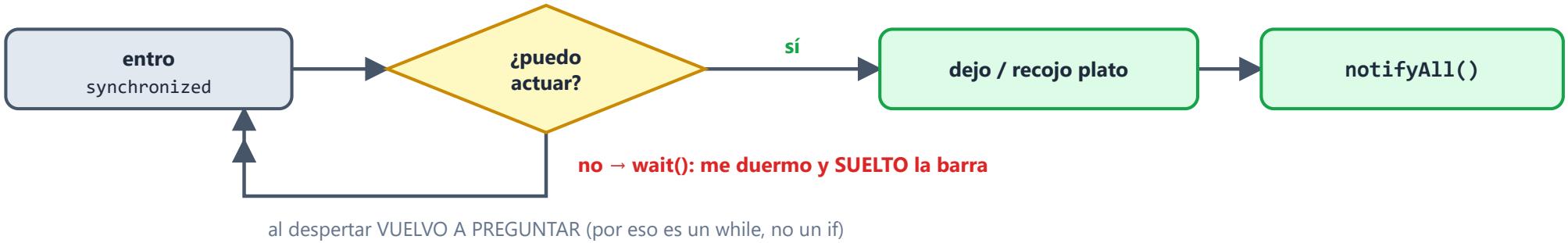
pág. 8 · C05

> Cómo estudiar este tema: **1_HISTORIA.md** con este PDF al lado → **3_PREGUNTAS.md** → **2_TECNICO.md** solo para consultar la sintaxis.

El cocinero, la barra y los camareros



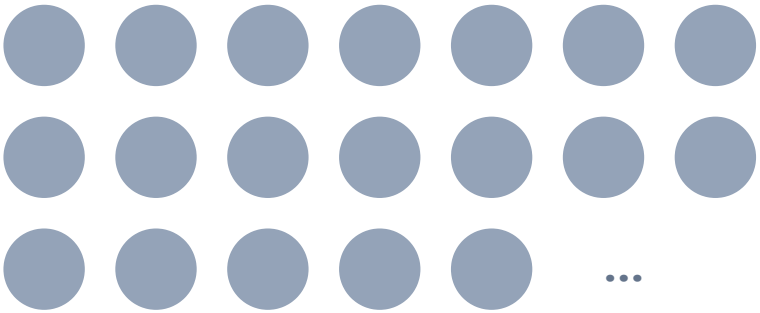
Lo que hace cada hilo al entrar (siempre dentro de synchronized)



> «Si no hay nada que hacer, me duermo y suelto la barra; cuando cambie algo, me avisan.» · ⚠ **while**, nunca if · wait/notify solo dentro de **synchronized**

No contrato un camarero por cliente

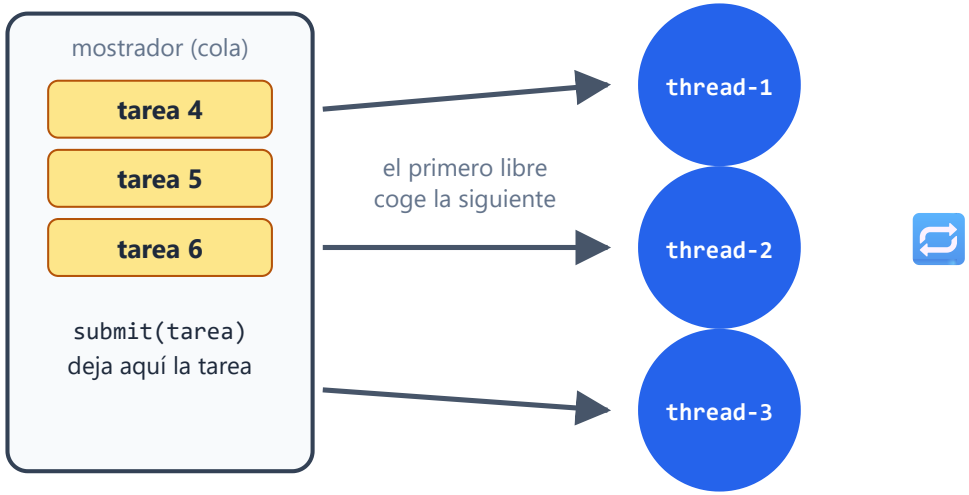
❌ Sin pool: un hilo por cliente



1000 clientes = 1000 `new Thread`

crear hilos es caro y satura el ordenador

✅ Con pool: plantilla fija de 3



Salida real de C02 (6 clientes, 3 hilos):

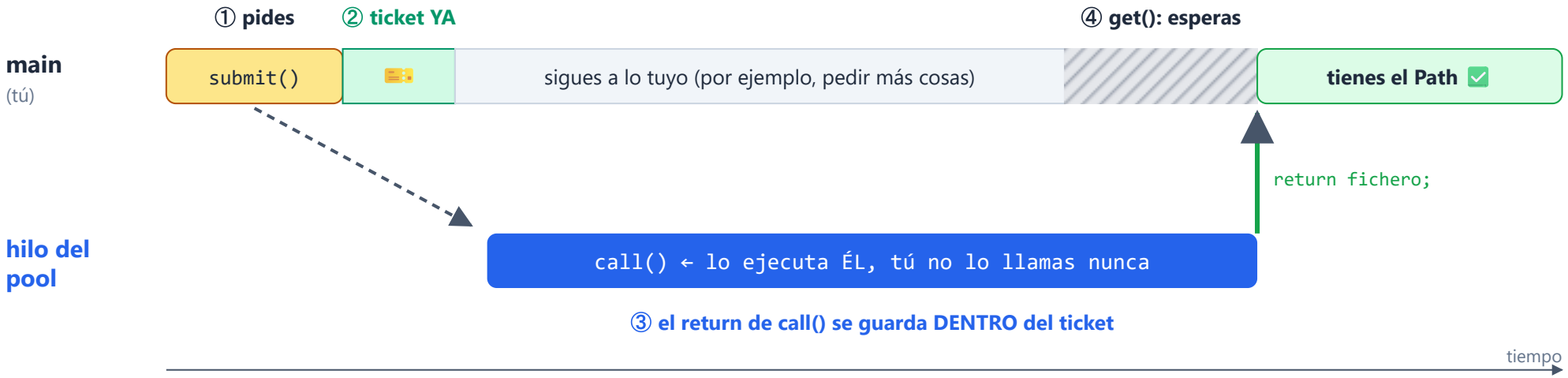
```
Cliente #2 atendido por pool-1-thread-2
Cliente #1 atendido por pool-1-thread-1
Cliente #3 atendido por pool-1-thread-3
Cliente #5 atendido por pool-1-thread-2   ← repite
Cliente #4 atendido por pool-1-thread-3   ← repite
Cliente #6 atendido por pool-1-thread-1   ← repite
```

La tarea NO es el camarero. `WriteCSV` es el **encargo**; los camareros son los hilos del pool. 5 encargos, 3 camareros.

Sin `shutdown()` el programa no termina: los camareros se quedan esperando clientes para siempre.

El ticket del pedido

Pido, me dan un ticket, sigo a lo mío… y con el ticket recojo el resultado cuando lo necesito.



El tipo viaja de punta a punta

```
Callable<Path>  → call() da Path
submit()        → Future<Path>
future.get()    → Path
```

Los errores también viajan en el ticket
Si `call()` falla, no lo ves… hasta el `get()`, que lanza **ExecutionException**. La causa real: `e.getCause()` / «Caused by:». (te pasó con LocalDate)

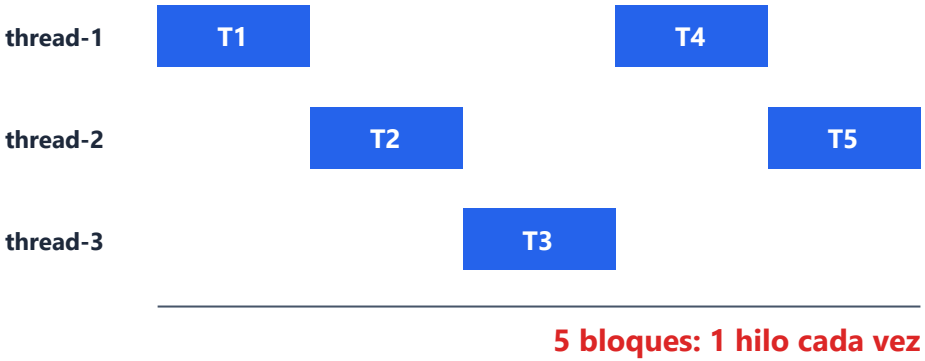
⑤ Al final, shutdown()
Recogidos todos los tickets, cierras la cafetería.

Run ↔ Callable: `start()` llama a `run()`; `submit()` hace que el pool llame a `call()`.

Primero repartir todo, después recoger todo

5 tareas, pool de 3 hilos. Cada tarea tarda lo mismo (1 bloque).

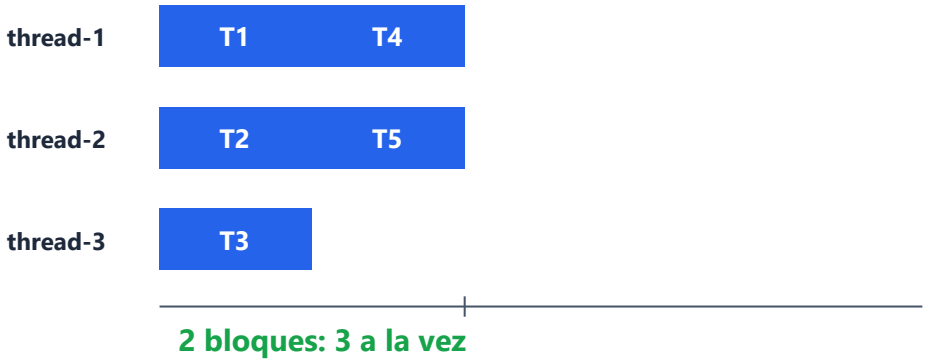
❌ get() dentro del bucle del submit



cada get() espera a que acabe ANTES de repartir la siguiente

```
// ❌ MAL
for (int i = 1; i <= 5; i++) {
    Future<Path> f = pool.submit(new WriteCSV(i));
    f.get();                // espera aquí → serializa
}
```

✅ dos bucles separados



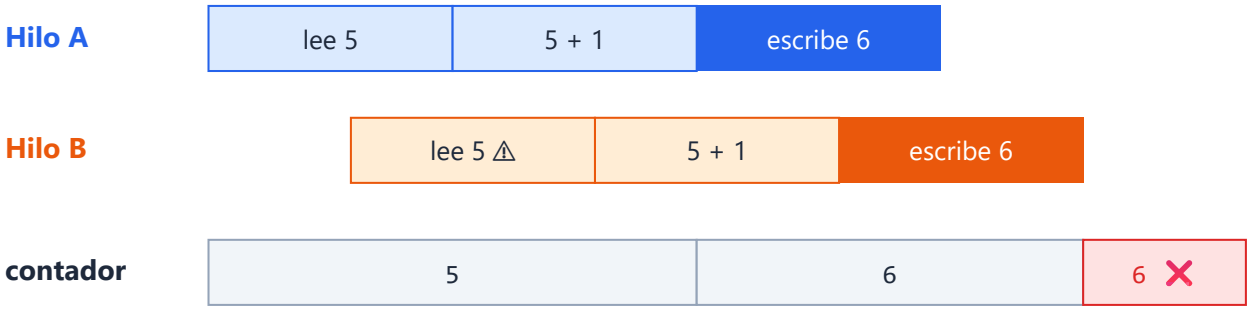
las 5 se reparten al momento; luego se recogen

```
// ✅ BIEN
for (int i = 1; i <= 5; i++)                // BUCLE 1: repartir
    futures.add(pool.submit(new WriteCSV(i)));
for (Future<Path> future : futures) {       // BUCLE 2: recoger
    Path ruta = future.get();
    mostrarInfo(ruta);
}
pool.shutdown();
```

> En la llamada se pasa **la variable**: `mostrarRutas(futures)` ✅ — nunca el tipo: `mostrarRutas(Path)` ❌

El contador que pierde cuentas

`contador++` parecen un paso, pero son tres: leer → sumar → escribir.



2 sumas → solo +1

se ha perdido un café

C03: esperado 400 000 → int: 307 769

AtomicInteger: 400 000 ✅

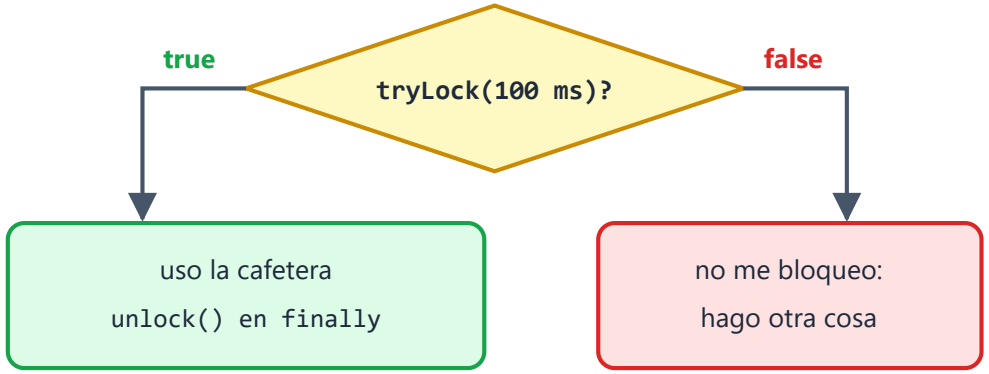
AtomicInteger = leer + sumar + escribir **de un golpe**.

```
contador.incrementAndGet();
```

¿Cuál uso?

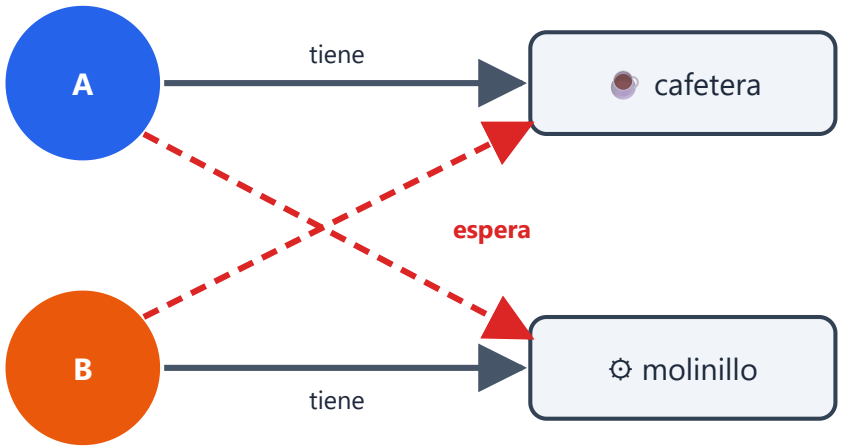
Un contador o valor simple	`AtomicInteger`
Un bloque de código normal	`synchronized`
tryLock / tiempo máximo / fairness	`ReentrantLock`

ReentrantLock: el cerrojo con paciencia limitada



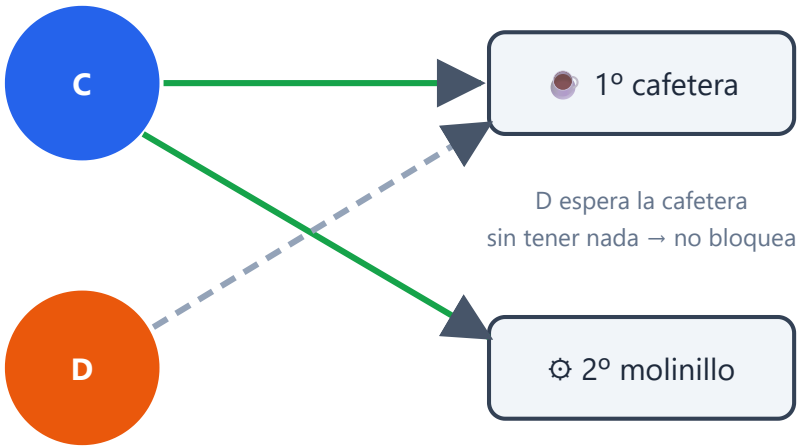
Dos camareros esperándose para siempre

❌ **Orden contrario → círculo**



C04: «⚠ INTERBLOQUEO… A=BLOCKED, B=BLOCKED»

✅ **Mismo orden → sin círculo**



C04: «✓ Los dos han preparado su café»

1. Exclusión mutua
la cafetera, de uno en uno

2. Retención y espera
tengo una y espero otra sin soltar

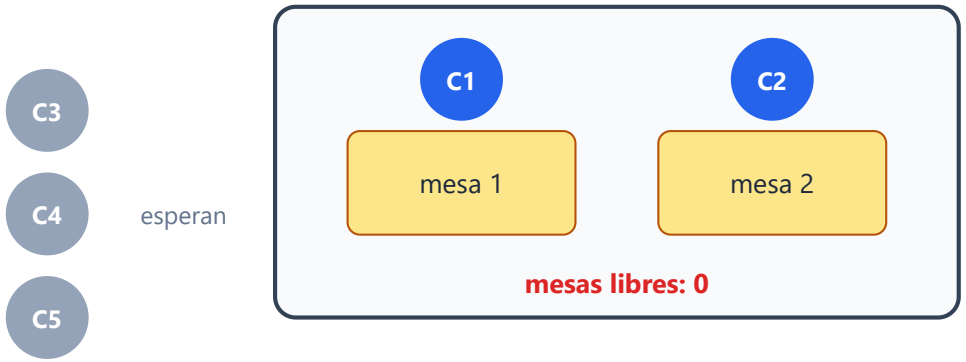
3. No expropiación
nadie me la quita

4. Espera circular
A→B→A · **rómpela con el mismo orden**

Se tienen que dar **las 4 a la vez**: basta con romper una. Y ojo: el programa se congela **sin ningún error**.

Solo caben N · No empiezo hasta que lleguen todos

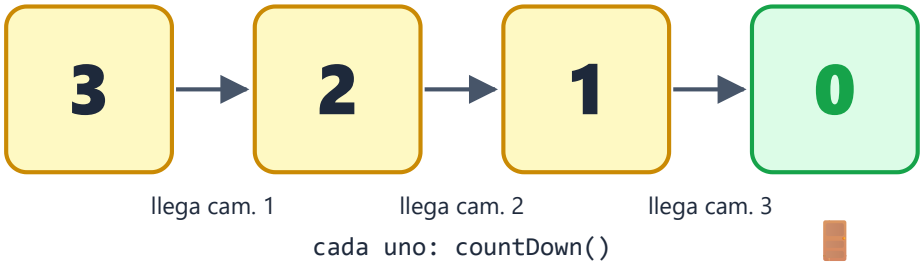
🪑 Semaphore(2): 5 clientes, 2 mesas




```
acquire() → coger mesa (espera si no hay)
release() → dejarla (en finally)
```

nunca hay más de 2 sentados

⏳ CountDownLatch(3): abrir cuando estén todos




```
await() → el encargado espera hasta 0
```

«✓ Están todos: ¡abrimos la cafetería!»

de un solo uso: no se reinicia

	join()	CountDownLatch
Espera a…	que un hilo **termine**	que ocurra un **evento** N veces (los hilos pueden seguir vivos)

¿Qué herramienta uso?

Mi problema es...	Herramienta	Frase clave	Error típico
Esperar a que haya algo (o sitio)	`wait / notifyAll`	Me duermo, suelto y me avisan	`if` en vez de `while`
Muchas tareas sin crear mil hilos	`ExecutorService`	Plantilla fija que se turna	Olvidar `shutdown()`
Que la tarea me devuelva algo	`Callable + Future`	Ticket del pedido	`get()` en el bucle del `submit` · llamar tú a `call()`
Un contador compartido	`AtomicInteger`	Sumar de un golpe	Usar `int` + `++`
Cerrojo con tryLock / tiempo máximo	`ReentrantLock`	Paciencia limitada	`unlock()` fuera del `finally`
Que no se bloqueen para siempre	Mismo orden	Todos en el mismo orden	Coger recursos en orden distinto
Solo N a la vez	`Semaphore`	Solo caben N	`release()` fuera del `finally`
Esperar a que lleguen todos	`CountDownLatch`	Cuenta atrás	Querer reutilizarlo

Los que se sueltan solos
`synchronized` (también con `wait`)

Los que sueltas TÚ (finally)
`ReentrantLock.unlock()`, `Semaphore.release()`

Los que se cierran
el pool: `shutdown()`